



Informatik II – Machine Learning

Informatik II – Woche 1

12. / 13. Mai 2026

Heutiges Programm

Lineare Regression

Programmierübung

k-Nearest Neighbors

Cross-Validation

Logistische Regression

1. Lineare Regression

Was ist Regression?

- **Regression:** aus Merkmalen einen kontinuierlichen Wert vorhersagen.

Was ist Regression?

- **Regression:** aus Merkmalen einen kontinuierlichen Wert vorhersagen.
- **Lineare Regression:** die Vorhersage erfolgt über eine lineare Abbildung.

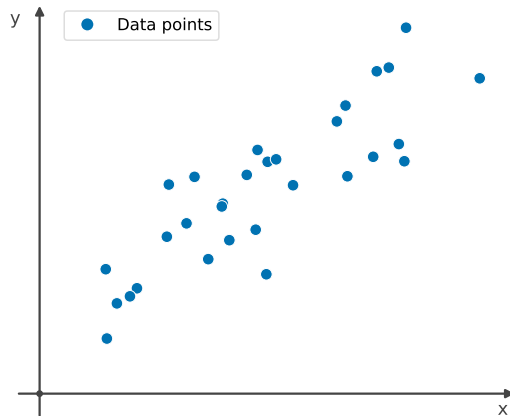
Was ist Regression?

- **Regression:** aus Merkmalen einen kontinuierlichen Wert vorhersagen.
- **Lineare Regression:** die Vorhersage erfolgt über eine lineare Abbildung.
- Geometrisch: eine Gerade (oder Hyperebene) durch die Daten legen.

Der Datensatz

Merkmale $\mathbf{x}_i$		Label y_i
Lern h	Schlaf h	Punkte
2	7	58
5	6	74
8	8	91
$\vdots$	$\vdots$	$\vdots$

$$\mathbf{x}_i \in \mathbb{R}^D, \quad y_i \in \mathbb{R}, \quad i = 1, \dots, n.$$



ein Merkmal ($D = 1$) zur Visualisierung dargestellt

Das Modell

$$\hat{y} = w_0 + w_1 x$$

Ein Merkmal: eine Gerade mit Steigung w_1 und Achsenabschnitt w_0 .

(Standard Definition einer Lin. Funktion)

Das Modell

$$f(x) = mx + q$$

— "offset"



$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_D x_D$$

D Merkmale: jedes w_j ist das Gewicht des Merkmals x_j .

Das Modell

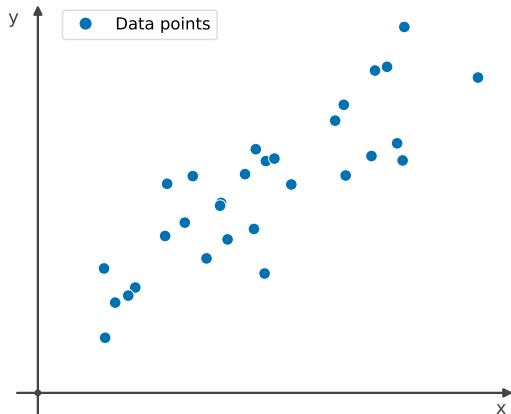
$$\hat{y} = \mathbf{w} \cdot \mathbf{x}$$

Kompakte Form: man stellt der $\mathbf{x}$ eine 1 voran, damit der Bias w_0 mit ihr multipliziert wird.

$$\mathbf{w} = (w_0, w_1, \dots, w_D), \quad \mathbf{x} = (1, x_1, \dots, x_D).$$

Beispiel: $n = 30, D = 1$

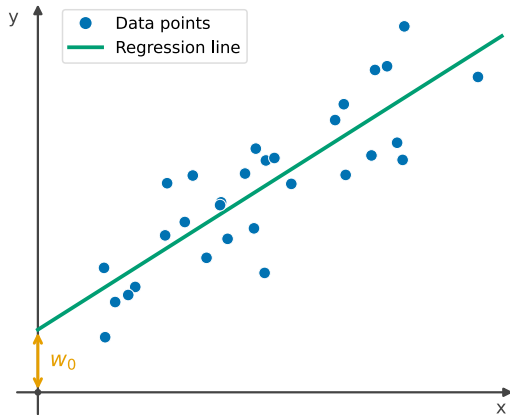
bsp. Note



bsp. Leshaufwand

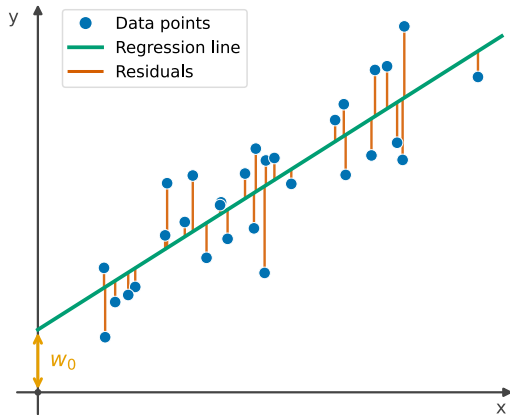
- Die Daten: ein Merkmal auf der x -Achse, das Label auf der y -Achse.

Beispiel: $n = 30, D = 1$



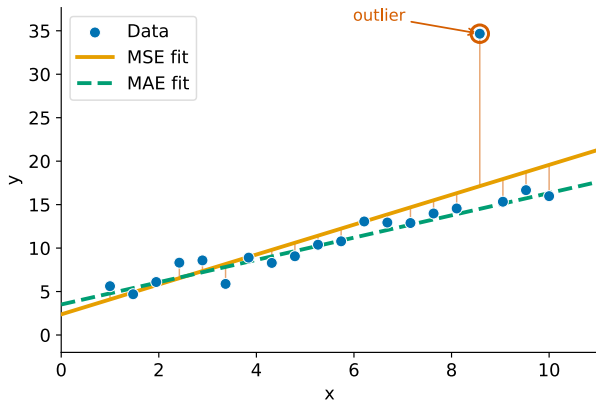
- Die Daten: ein Merkmal auf der x -Achse, das Label auf der y -Achse.
- Die Anpassung. w_0 ist der y -Achsenabschnitt.

Beispiel: $n = 30, D = 1$



- Die Daten: ein Merkmal auf der x -Achse, das Label auf der y -Achse.
- Die Anpassung. w_0 ist der y -Achsenabschnitt.
- Die Residuen $\hat{y}_i - y_i$, genau das, was der Verlust misst.

Verlustfunktion



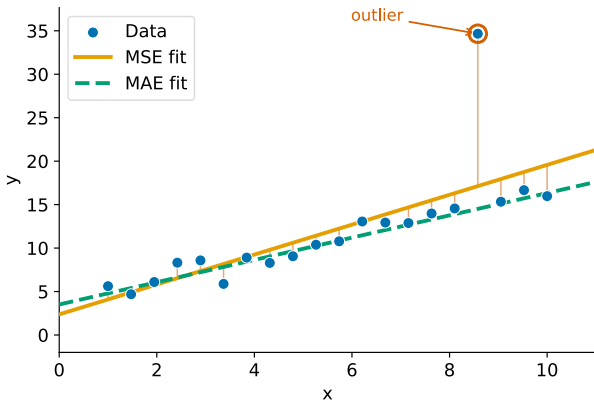
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Verlustfunktion: Eine Metrik, welche unsere "Performance" des Modells/Regression misst. Und anhand dessen sich das Modell selbst austariert (fitten);

oder wir das Modell selbst abändern (Hyperparameter)

Verlustfunktion



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

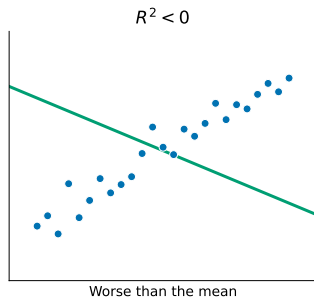
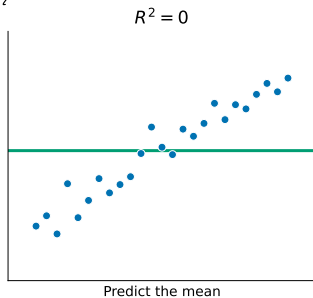
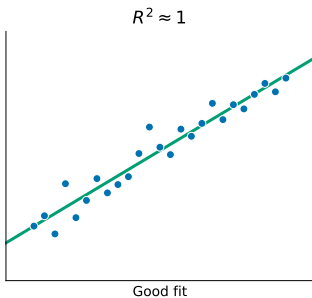
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

MSE quadriert die Fehler, sodass ein einzelner **Ausreisser** die Gerade an sich zieht. MAE ist robust. Allerdings behandelt der MAE alle Fehler gleich, wodurch grosse Abweichungen, die in manchen Anwendungen von Bedeutung sind, möglicherweise untergewichtet werden.

Die R^2 -Kennzahl

$$R^2 = 1 - \frac{\sum_i (\overset{\text{Original}}{y_i} - \overset{\text{predicted}}{\hat{y}_i})^2}{\sum_i (\overset{\text{Original}}{y_i} - \bar{y})^2}$$

vergleicht das Modell mit der Mittelwert-Basislinie



Anders als der Verlust ist R^2 einheitslos und zwischen Datensätzen vergleichbar.

Linear in den Gewichten

„Linear“ bezieht sich auf die **Gewichte**, *nicht* auf die Eingabe.

Linear in den Gewichten

„Linear“ bezieht sich auf die Gewichte, *nicht* auf die Eingabe.

Gekrümmte Daten? Einfach neue Merkmale hinzufügen:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

weiterhin lineare Regression, nur mit $\mathbf{x} = (1, x, x^2)$.

Linear in den Gewichten

„Linear“ bezieht sich auf die Gewichte, *nicht* auf die Eingabe.

Gekrümmte Daten? Einfach neue Merkmale hinzufügen:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

weiterhin lineare Regression, nur mit $\mathbf{x} = (1, x, x^2)$.

Derselbe Trick passt Parabeln (geworfener Ball), v^2 (Bremsweg), $\log x$, $\sin x$, ...

Zusammenfassung

Lineare Regression passt eine Hyperebene an, indem sie den quadratischen Fehler minimiert.

„Linear“ bezieht sich auf die Gewichte,
nicht auf die Merkmale.

2. Theoretische Übungen

Übung 1: Wahl der Loss-Funktion

Gegeben sei eine lineare Regression zur Vorhersage von Hauspreisen. Ein Datenpunkt zeigt einen Preis von \$10,000,000, vermutlich ein Tippfehler: er sollte wohl \$1,000,000 lauten.

Zur Wahl stehen zwei Verlustfunktionen:

- Mittlerer quadratischer Fehler (MSE): $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$
- Mittlerer absoluter Fehler (MAE): $\frac{1}{n} \sum_i |\hat{y}_i - y_i|$

Übung 1: Wahl der Loss-Funktion

Gegeben sei eine lineare Regression zur Vorhersage von Hauspreisen. Ein Datenpunkt zeigt einen Preis von \$10,000,000, vermutlich ein Tippfehler: er sollte wohl \$1,000,000 lauten.

Zur Wahl stehen zwei Verlustfunktionen:

- Mittlerer quadratischer Fehler (MSE): $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$
- Mittlerer absoluter Fehler (MAE): $\frac{1}{n} \sum_i |\hat{y}_i - y_i|$

Welche ist vorzuziehen, und warum?

Übung 1: Wahl der Loss-Funktion

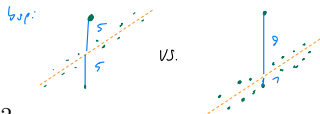
Gegeben sei eine lineare Regression zur Vorhersage von Hauspreisen. Ein Datenpunkt zeigt einen Preis von \$10,000,000, vermutlich ein Tippfehler: er sollte wohl \$1,000,000 lauten.

Zur Wahl stehen zwei Verlustfunktionen:

- Mittlerer quadratischer Fehler (MSE): $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$
- Mittlerer absoluter Fehler (MAE): $\frac{1}{n} \sum_i |\hat{y}_i - y_i|$

Welche ist vorzuziehen, und warum?

❗ **MAE.** Der MSE quadriert den Fehler dieses einen Ausreissers, sodass ein einzelner weit entfernter Punkt die gesamte Anpassung zu sich ziehen kann. Der MAE bestraft Fehler linear und ist daher deutlich robuster gegenüber Ausreissern.



Übung 2: Gewichte interpretieren

Gegeben sei eine lineare Regression zur Vorhersage der Prüfungspunkte (von 100). Alle Merkmale sind ähnlich skaliert, das Modell lautet:

$$\hat{y} = 50 + 5 \cdot \text{Lernstd.} - 3 \cdot \text{Handystd.} + 2 \cdot \text{Schlafstd.}$$

Übung 2: Gewichte interpretieren

Gegeben sei eine lineare Regression zur Vorhersage der Prüfungspunkte (von 100). Alle Merkmale sind ähnlich skaliert, das Modell lautet:

$$\hat{y} = 50 + 5 \cdot \text{Lernstd.} - 3 \cdot \text{Handystd.} + 2 \cdot \text{Schlafstd.}$$

- (a) Welches Merkmal hat den stärksten *negativen* Einfluss?
- (b) Eine Stunde mehr Lernen (sonst gleich): wie ändert sich die Vorhersage?
- (c) Was bedeutet die Konstante 50?

Übung 2: Gewichte interpretieren

Gegeben sei eine lineare Regression zur Vorhersage der Prüfungspunkte (von 100). Alle Merkmale sind ähnlich skaliert, das Modell lautet:

$$\hat{y} = 50 + 5 \cdot \text{Lernstd.} - 3 \cdot \text{Handystd.} + 2 \cdot \text{Schlafstd.}$$

- (a) Welches Merkmal hat den stärksten *negativen* Einfluss?
- (b) Eine Stunde mehr Lernen (sonst gleich): wie ändert sich die Vorhersage?
- (c) Was bedeutet die Konstante 50?

- ❗ (a) Handystd., mit Gewicht -3 .
- (b) Die Vorhersage steigt um 5 Punkte.
- (c) Der Bias w_0 : die Vorhersage bei Wert 0 für alle Merkmale.

Übung 3: Lineare Regression für nichtlineare Daten

Gegeben seien Messungen des Bremswegs eines Autos bei verschiedenen Geschwindigkeiten. Aufgetragen, zeigen die Daten eine deutlich aufwärts gekrümmte Kurve; eine Gerade passt schlecht.

- (a) Liefert eine lineare Anpassung $\hat{y} = w_0 + w_1 \cdot v$ gute Resultate?
- (b) Kann lineare Regression die Daten dennoch modellieren? Falls ja, wie?

Übung 3: Lineare Regression für nichtlineare Daten

Gegeben seien Messungen des Bremswegs eines Autos bei verschiedenen Geschwindigkeiten. Aufgetragen, zeigen die Daten eine deutlich aufwärts gekrümmte Kurve; eine Gerade passt schlecht.

- (a) Liefert eine lineare Anpassung $\hat{y} = w_0 + w_1 \cdot v$ gute Resultate?
- (b) Kann lineare Regression die Daten dennoch modellieren? Falls ja, wie?

❗ (a) Nein, eine Gerade erfasst die Krümmung nicht.

(b) Ja. Man nimmt v^2 als zusätzliches Merkmal: $\hat{y} = w_0 + w_1 v + w_2 v^2$. Linear in den Gewichten w_0, w_1, w_2 , obwohl quadratisch in v . Physikalisch passt das: der Bremsweg ist proportional zur kinetischen Energie $\propto v^2$.

3. Programmierübung

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

<https://expert.ethz.ch/enrolled/SS26/mavt2/exercises>

■ **Diabetes Prediction** , Homework

4. k-Nearest Neighbors

k-Nearest Neighbors

Die Idee in einem Satz:

k-Nearest Neighbors

Die Idee in einem Satz:

*Um einen neuen Punkt zu klassifizieren, betrachtet man die **k** nächsten Trainingspunkte und lässt sie **abstimmen**.*

k-Nearest Neighbors

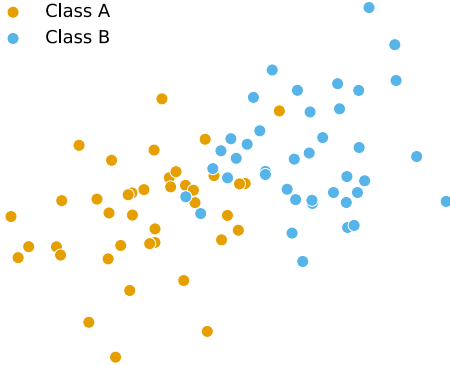
Die Idee in einem Satz:

*Um einen neuen Punkt zu klassifizieren, betrachtet man die **k** nächsten Trainingspunkte und lässt sie **abstimmen**.*

Das ist alles. Es gibt keine Gleichung zu lösen, keine Parameter zu fitten. Das Modell ist schlicht die gesamte Trainingsmenge.

k-Nearest Neighbors: Der Algorithmus

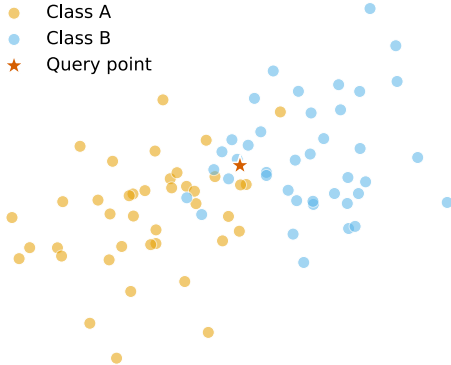
● Class A
● Class B



■ Gelabelte Trainingsmenge
mit zwei Klassen (orange,
blau).

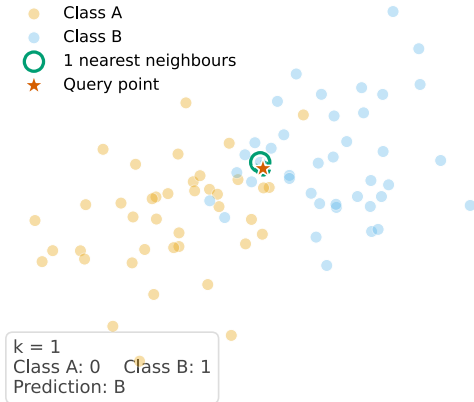
k-Nearest Neighbors: Der Algorithmus

- Class A
- Class B
- ★ Query point



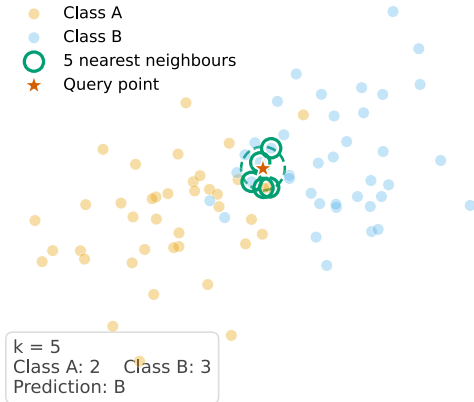
- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer Anfragepunkt: welche Klasse?

k-Nearest Neighbors: Der Algorithmus



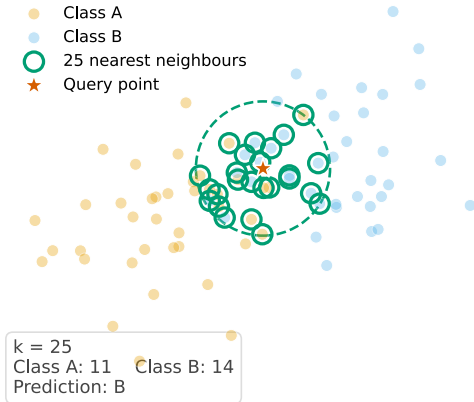
- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer **Anfragepunkt**: welche Klasse?
- $k = 1$: nächster Trainingspunkt entscheidet. Stimme: blau.

k-Nearest Neighbors: Der Algorithmus



- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer **Anfragepunkt**: welche Klasse?
- $k = 1$: nächster Trainingspunkt entscheidet. Stimme: blau.
- $k = 5$: die fünf nächsten. Mehrheitsentscheid.

k-Nearest Neighbors: Der Algorithmus



- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer **Anfragepunkt**: welche Klasse?
- $k = 1$: nächster Trainingspunkt entscheidet. Stimme: blau.
- $k = 5$: die fünf nächsten. Mehrheitsentscheid.
- $k = 25$: fünfundzwanzig. Das Ergebnis kann kippen!

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.
2. Man wählt die k Trainingspunkte mit der kleinsten Distanz; diese Menge nennt man $\mathcal{N}_k(\mathbf{x})$.

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.
2. Man wählt die k Trainingspunkte mit der kleinsten Distanz; diese Menge nennt man $\mathcal{N}_k(\mathbf{x})$.
3. Man gibt das häufigste Label unter ihnen zurück:

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]$$

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.
2. Man wählt die k Trainingspunkte mit der kleinsten Distanz; diese Menge nennt man $\mathcal{N}_k(\mathbf{x})$.
3. Man gibt das häufigste Label unter ihnen zurück:

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]$$

oder 0

- Für die **Regression**: man gibt den Mittelwert der k Nachbarwerte zurück statt eines Mehrheitsentscheids. (kein Arg Max)

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

- Lineare Regression (wie bereits gesehen) und logistische Regression (wie später noch zu sehen): Training ist teuer, Vorhersage ist günstig (ein Skalarprodukt).

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

- Lineare Regression (wie bereits gesehen) und logistische Regression (wie später noch zu sehen): Training ist teuer, Vorhersage ist günstig (ein Skalarprodukt).
- k-NN: Training ist *gratis* (nur die Daten speichern), aber *jede* Vorhersage kostet $O(n \cdot D)$, weil man den ganzen Datensatz durchgehen muss.

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

- Lineare Regression (wie bereits gesehen) und logistische Regression (wie später noch zu sehen): Training ist teuer, Vorhersage ist günstig (ein Skalarprodukt).
 - k-NN: Training ist *gratis* (nur die Daten speichern), aber *jede* Vorhersage kostet $O(n \cdot D)$, weil man den ganzen Datensatz durchgehen muss.
- k-NN nennt man einen **lazy** oder **nicht-parametrischen** Lerner: er verschiebt die ganze Arbeit auf den Moment der Anfrage.

k-Nearest Neighbors: Welcher Abstand?

“Am nächsten” hängt davon ab, wie man die Distanz misst. Standardmässig verwendet man die **euklidische Distanz**:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^D (x_j - x'_j)^2}$$

k-Nearest Neighbors: Welcher Abstand?

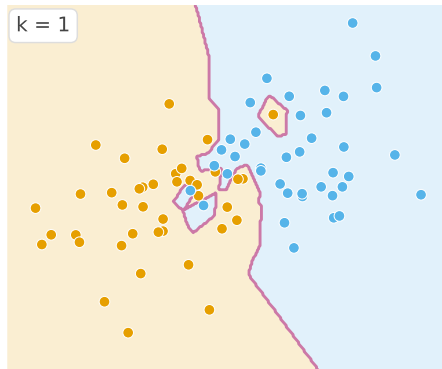
“Am nächsten” hängt davon ab, wie man die Distanz misst. Standardmässig verwendet man die **euklidische Distanz**:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^D (x_j - x'_j)^2}$$

Andere Wahlmöglichkeiten existieren, aber für diesen Kurs genügt es, die euklidische Distanz im Kopf zu haben. Man erinnere sich daran, dass die euklidische Distanz nichts anderes ist als die Länge der geraden Linie, die zwei Punkte im Raum verbindet.

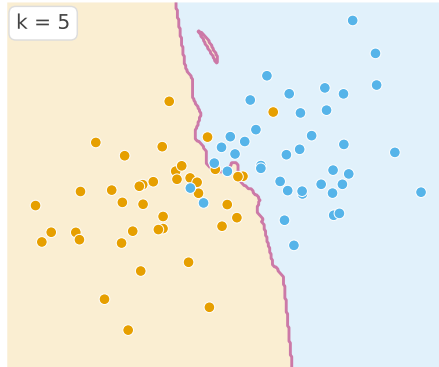
eukl. Norm: Standard "Betrag eines Vektors"

Wirkung von k : Entscheidungsgrenze



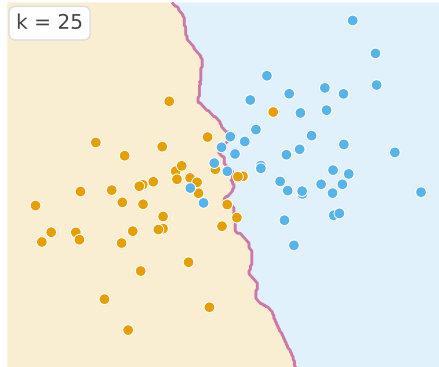
$k = 1$: die Entscheidungsgrenze legt sich um jeden Trainingspunkt, ist zackig und folgt dem Rauschen.

Wirkung von k : Entscheidungsgrenze



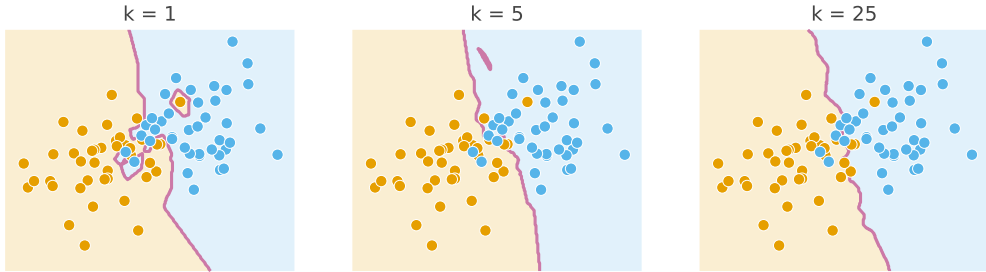
$k = 5$: glatter, ignoriert isolierte Ausreisser.

Wirkung von k : Entscheidungsgrenze

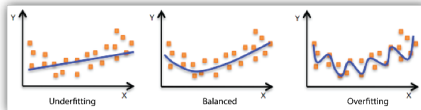


$k = 25$: sehr glatt, fast eine Gerade, verliert aber feine Details.

Wirkung von k : Entscheidungsgrenze



Im Vergleich: kleines k overfittet, grosses k underfittet (mehr dazu in der Trainings-Lektion).

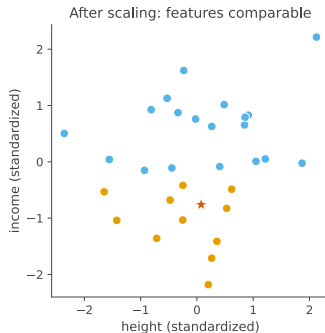
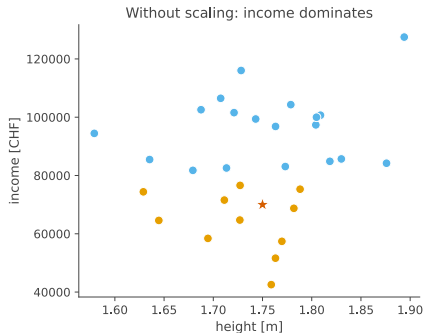


Vorsicht: Skalierung der Merkmale

k-NN nutzt rohe Distanzen: ein Merkmal mit grossem Wertebereich dominiert.

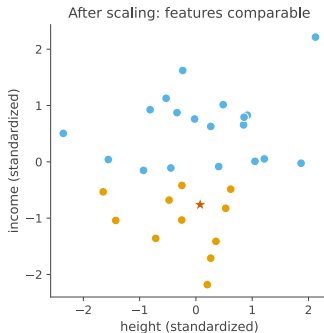
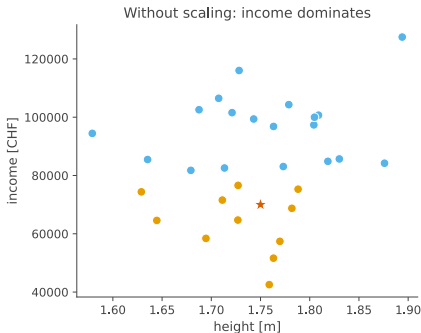
Vorsicht: Skalierung der Merkmale

k-NN nutzt rohe Distanzen: ein Merkmal mit grossem Wertebereich dominiert.



Vorsicht: Skalierung der Merkmale

k-NN nutzt rohe Distanzen: ein Merkmal mit grossem Wertebereich dominiert.



→ Vor dem Anwenden von k-NN die Merkmale stets **standardisieren**.

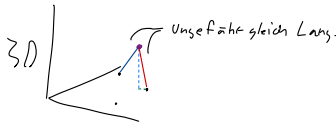
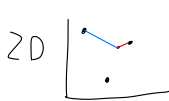
Zusammenfassung

Stärken

- Konzeptuell trivial.
- Keine Trainingskosten.
- Erfasst nicht-lineare Grenzen ohne Aufwand.
- Funktioniert für Klassifikation *und* Regression.

Schwächen

- Langsam zur Vorhersagezeit: $O(nD)$ pro Anfrage.
- Speichert den ganzen Datensatz.
- Empfindlich gegenüber Merkmalsskalierung.
- Leidet in hohen Dimensionen ("Fluch der Dimensionalität").



Zusammenfassung

Stärken

- Konzeptuell trivial.
- Keine Trainingskosten.
- Erfasst nicht-lineare Grenzen ohne Aufwand.
- Funktioniert für Klassifikation *und* Regression.

Schwächen

- Langsam zur Vorhersagezeit: $O(nD)$ pro Anfrage.
- Speichert den ganzen Datensatz.
- Empfindlich gegenüber Merkmalsskalierung.
- Leidet in hohen Dimensionen (“Fluch der Dimensionalität”).

k ist der einzige Stellknopf, ein **Hyperparameter**, wie in der Trainings-Lektion zu sehen sein wird

5. Theoretische Übungen

Übung 1: Klassifikation von Hand

Gegeben sind die folgenden gelabelten Trainingspunkte in $\mathbb{R}^2$:

i	x_1	x_2	y
1	1	1	A
2	2	1	A
3	4	3	B
4	5	4	B
5	3	4	B

Man klassifiziere den Anfragepunkt $\mathbf{x} = (3, 2)$ mit **(a)** $k = 1$ und **(b)** $k = 3$.

Übung 1: Klassifikation von Hand (Lösung)

Distanzen von $\mathbf{x} = (3, 2)$:

i	$d(\mathbf{x}, \mathbf{x}_i)$	y_i
1	$\sqrt{(3-1)^2 + (2-1)^2} = \sqrt{5} \approx 2.24$	A
2	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	A
3	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	B
4	$\sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83$	B
5	$\sqrt{0^2 + 2^2} = 2$	B

Übung 1: Klassifikation von Hand (Lösung)

Distanzen von $\mathbf{x} = (3, 2)$:

i	$d(\mathbf{x}, \mathbf{x}_i)$	y_i
1	$\sqrt{(3-1)^2 + (2-1)^2} = \sqrt{5} \approx 2.24$	A
2	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	A
3	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	B
4	$\sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83$	B
5	$\sqrt{0^2 + 2^2} = 2$	B

- ❗ (a) $k = 1$: Stichentscheid zwischen Punkt 2 (A) und Punkt 3 (B), beide mit Distanz $\sqrt{2}$. A oder B ist vertretbar; ein typischer Stichentscheid ist, den nächstfolgenden Nachbarn heranzuziehen.
- (b) $k = 3$: die drei nächsten sind 2, 3, 5 mit Labels A, B, B. Mehrheit: **B**.

Übung 2: k wählen

Man entscheide für jede Aussage, ob sie **wahr** oder **falsch** ist, und begründe kurz.

- (c) Wenn k gleich der Grösse der Trainingsmenge ist, ist die Vorhersage für jeden Anfragepunkt dieselbe.
- (d) k -NN kann eine nicht-lineare Entscheidungsgrenze lernen, obwohl es keine lernbaren Parameter besitzt.

Übung 2: k wählen

Man entscheide für jede Aussage, ob sie **wahr** oder **falsch** ist, und begründe kurz.

- (c) Wenn k gleich der Grösse der Trainingsmenge ist, ist die Vorhersage für jeden Anfragepunkt dieselbe.
- (d) k -NN kann eine nicht-lineare Entscheidungsgrenze lernen, obwohl es keine lernbaren Parameter besitzt.
- (c) **Wahr.** Die Vorhersage ist die Mehrheitsklasse der gesamten Trainingsmenge.
- (d) **Wahr.** Mit genügend Daten kann die Grenze jede Form annehmen.

Übung 3: Eine Falle bei der Merkmalsskalierung

Man baut einen k-NN-Klassifikator, um Fahrräder zu empfehlen. Jede Kundin und jeder Kunde wird durch *Körpergrösse* (Meter, Bereich ~ 1.5 – 2.0) und *Jahreseinkommen* (CHF, Bereich $\sim 30,000$ – $200,000$) beschrieben.

Ohne jegliche Vorverarbeitung trainiert man k-NN mit $k = 5$ und stellt fest, dass das Modell die Körpergrösse vollständig ignoriert.

Übung 3: Eine Falle bei der Merkmalsskalierung

Man baut einen k-NN-Klassifikator, um Fahrräder zu empfehlen. Jede Kundin und jeder Kunde wird durch *Körpergrösse* (Meter, Bereich $\sim 1.5\text{--}2.0$) und *Jahreseinkommen* (CHF, Bereich $\sim 30,000\text{--}200,000$) beschrieben.

Ohne jegliche Vorverarbeitung trainiert man k-NN mit $k = 5$ und stellt fest, dass das Modell die Körpergrösse vollständig ignoriert.

(a) Wieso passiert das?

(b) Welcher einzige Vorverarbeitungsschritt behebt das?

Übung 3: Eine Falle bei der Merkmalsskalierung

Man baut einen k-NN-Klassifikator, um Fahrräder zu empfehlen. Jede Kundin und jeder Kunde wird durch *Körpergrösse* (Meter, Bereich ~ 1.5 – 2.0) und *Jahreseinkommen* (CHF, Bereich $\sim 30,000$ – $200,000$) beschrieben.

Ohne jegliche Vorverarbeitung trainiert man k-NN mit $k = 5$ und stellt fest, dass das Modell die Körpergrösse vollständig ignoriert.

(a) Wieso passiert das?

(b) Welcher einzige Vorverarbeitungsschritt behebt das?

❗ (a) Die Distanzen werden vom Einkommen dominiert: eine Differenz von CHF 10,000 trägt $10,000^2$ zur quadrierten Distanz bei, jede Differenz in der Körpergrösse weniger als 1. Die Körpergrösse ist unsichtbar.

(b) Die Merkmale **standardisieren**, damit sie in vergleichbaren Bereichen liegen.

6. Cross-Validation

Warum brauchen wir Validierung?

Ziel: Wir wollen abschätzen, wie gut ein Modell auf ungesehenen Daten abschneidet.

- Die Trainings-Performance ist meist zu optimistisch.
- Ein Modell kann die Trainingsdaten auswendig lernen.
- Deshalb brauchen wir Daten, die nicht für das Anpassen des Modells verwendet wurden.

Validierung hilft uns, die Generalisierungs-Performance abzuschätzen.

Das Problem mit einem einzelnen Split

Wenn wir die Daten nur einmal aufteilen, kann das Ergebnis stark vom Zufall abhängen.



Gleiches Modell, gleiche Daten, anderer Split, andere Schätzung.

k-Fold Cross-Validation

Idee: Verwende jeden Teil der Daten einmal zur Validierung.

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	
Lauf 1	val	train	train	train	train	88%
Lauf 2	train	val	train	train	train	91%
Lauf 3	train	train	val	train	train	86%
Lauf 4	train	train	train	val	train	90%
Lauf 5	train	train	train	train	val	89%

- Teile die Daten in k Folds auf.
- Trainiere auf $k - 1$ Folds.
- Validiere auf dem verbleibenden Fold.
- Wiederhole, bis jeder Fold einmal zur Validierung verwendet wurde.

Wie werten wir das Ergebnis aus?

Nach der k -fold Cross-Validation erhalten wir k Validierungs-Scores:

$$s_1, s_2, \dots, s_k$$

Üblicherweise berichten wir den mittleren Validierungs-Score:

$$\bar{s} = \frac{1}{k} \sum_{i=1}^k s_i$$

- Der Mittelwert schätzt die erwartete Performance.
- Die Streuung zwischen den Scores zeigt, wie stabil das Modell ist.

Hyperparameter wählen

Cross-Validation wird häufig zur Wahl von Hyperparametern verwendet.

Beispiel für kNN:

$$k_{\text{kNN}} \in \{1, 3, 5, 7, 9\}$$

- Probiere mehrere Hyperparameter-Werte aus.
- Berechne für jeden Wert den Cross-Validation-Score.
- Wähle den Wert mit der besten mittleren Validierungs-Performance.

Wichtig: Die finale Testmenge sollte erst ganz am Ende verwendet werden.

Praktische Details

- Typische Wahl: $k = 5$ oder $k = 10$.
 - Trainings- und Validierungs-Folds sind stets disjunkt.
 - Der Split erfolgt üblicherweise zufällig, aber ohne Zurücklegen.
- Set train

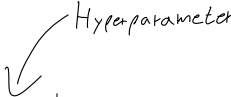
Spezialfall: Leave-One-Out Cross-Validation

- $k = n$, wobei n die Anzahl der Datenpunkte ist.
- Jeder Datenpunkt wird einmal als Validierungsmenge verwendet.
- Kann bei grossen Datensätzen rechenintensiv sein.

7. Theoretische Übungen

Aufgabe: k für kNN wählen

Wir führen eine 5-fold Cross-Validation auf einem kNN-Classifer für drei Kandidatenwerte von k durch. Die Validierungs-Genauigkeiten pro Fold lauten:



k_{kNN}	s_1	s_2	s_3	s_4	s_5
1	78%	82%	75%	80%	80%
3	86%	88%	84%	87%	85%
5	84%	85%	83%	86%	82%

Aufgaben:

- Berechne die mittlere Validierungs-Genauigkeit $\bar{s}$ für jeden Kandidaten.
- Welchen Wert von k_{kNN} sollten wir wählen?
- Wie viele Modell-Trainings hat die Cross-Validation insgesamt durchgeführt?

Lösung: k für kNN wählen

Mittlere Validierungs-Genauigkeit pro Kandidat:

$$\bar{s}_{k=1} = \frac{78+82+75+80+80}{5}\% = 79\%$$

$$\bar{s}_{k=3} = \frac{86+88+84+87+85}{5}\% = 86\%$$

$$\bar{s}_{k=5} = \frac{84+85+83+86+82}{5}\% = 84\%$$

Wähle $k_{\text{kNN}} = 3$, denn es hat die höchste mittlere Validierungs-Genauigkeit.

Gesamtanzahl Trainings während der CV: $G \cdot k = 3 \cdot 5 = 15$ Trainings.

Anschliessend trainiert man einmalig auf den vollen Trainingsdaten mit $k_{\text{kNN}} = 3$ und wertet dann genau einmal auf der Testmenge aus.

8. Logistische Regression

Warum logistische Regression?

Lineare Regression	Logistische Regression
sagt ein kontinuierliches y voraus	sagt eine Klassenwahrscheinlichkeit voraus
z. B. Preis, Temperatur, Punktzahl	z. B. Spam? Betrug? Krebs?
Ausgabe: eine beliebige reelle Zahl	Ausgabe: eine Zahl in $[0, 1]$

Warum logistische Regression?

Lineare Regression	Logistische Regression
sagt ein kontinuierliches y voraus	sagt eine Klassenwahrscheinlichkeit voraus
z. B. Preis, Temperatur, Punktzahl	z. B. Spam? Betrug? Krebs?
Ausgabe: eine beliebige reelle Zahl	Ausgabe: eine Zahl in $[0, 1]$

Gleiche Idee (ein lineares Modell), andere Frage (eine Ja/Nein-Antwort).

Der Datensatz

Beispiel. Vorhersagen, ob eine E-Mail Spam ist.

x_i (Merkmale)		y_i (Label)	
"\$\$\$ free!"	Links		
1	4	1	(Spam)
0	1	0	(kein Spam)
0	0	0	(kein Spam)

Der Datensatz

Beispiel. Vorhersagen, ob eine E-Mail Spam ist.

$\mathbf{x}_i$ (Merkmale)		y_i (Label)	
"\$\$\$ free!"	Links		
1	4	1	(Spam)
0	1	0	(kein Spam)
0	0	0	(kein Spam)

Gegeben sind n Datenpunkte. Jeder ist ein Merkmalsvektor $\mathbf{x}_i \in \mathbb{R}^D$ zusammen mit einem binären Label $y_i \in \{0, 1\}$.

Vektor

(anstatt räumliche Dimensionen (x, y, z) sind es $(\text{$$$ free, Links})$)

Zeilen!

Der Datensatz

Beispiel. Vorhersagen, ob eine E-Mail Spam ist.

$\mathbf{x}_i$ (Merkmale)		y_i (Label)	
"\$\$\$ free!"	Links		
1	4	1	(Spam)
0	1	0	(kein Spam)
0	0	0	(kein Spam)

Gegeben sind n Datenpunkte. Jeder ist ein Merkmalsvektor $\mathbf{x}_i \in \mathbb{R}^D$ zusammen mit einem binären Label $y_i \in \{0, 1\}$. **Ziel.** Eine Funktion finden, die die Wahrscheinlichkeit $\hat{y}_i \in [0, 1]$ vorhersagt, dass das Label 1 ist.

Von einer Zahl zu einer Wahrscheinlichkeit

Schritt 1. Linearen Score berechnen, wie bei der linearen Regression:

$$z = \mathbf{w} \cdot \mathbf{x}$$

- Problem: z kann *jede* reelle Zahl sein, eine Wahrscheinlichkeit muss aber in $[0, 1]$ liegen.

Von einer Zahl zu einer Wahrscheinlichkeit

Schritt 1. Linearen Score berechnen, wie bei der linearen Regression:

$$z = \mathbf{w} \cdot \mathbf{x}$$

→ Problem: z kann *jede* reelle Zahl sein, eine Wahrscheinlichkeit muss aber in $[0, 1]$ liegen.

Schritt 2. z durch die **Sigmoidfunktion** σ quetschen, um in $[0, 1]$ zu landen:

$$\hat{y} = \sigma(z) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

Von einer Zahl zu einer Wahrscheinlichkeit

Schritt 1. Linearen Score berechnen, wie bei der linearen Regression:

$$z = \mathbf{w} \cdot \mathbf{x}$$

→ Problem: z kann *jede* reelle Zahl sein, eine Wahrscheinlichkeit muss aber in $[0, 1]$ liegen.

Schritt 2. z durch die **Sigmoidfunktion** σ quetschen, um in $[0, 1]$ zu landen:

$$\hat{y} = \sigma(z) = \sigma(\mathbf{w} \cdot \mathbf{x})$$

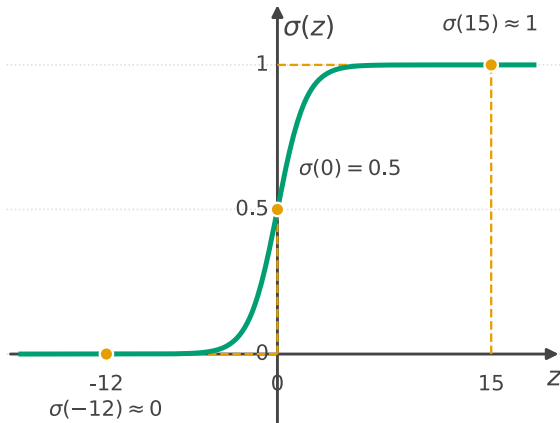
Logistische Regression = lineares Modell +
Sigmoid-Quetsche.

Die Sigmoidfunktion

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

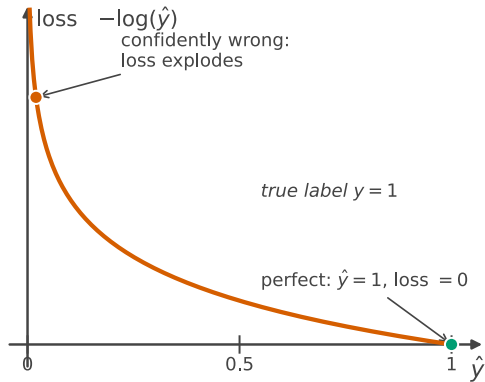
- grosses positives $z \rightarrow \hat{y} \approx 1$
- $z = 0 \rightarrow \hat{y} = 0.5$
- grosses negatives $z \rightarrow \hat{y} \approx 0$

Gross positiv: sicheres Ja.
Gross negativ: sicheres Nein.
Um 0 herum: unsicher.



Kreuzentropie-Verlust

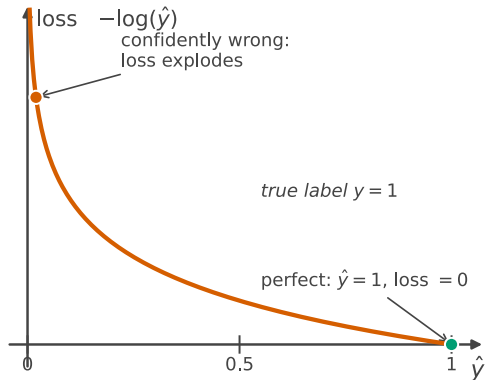
Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.



Kreuzentropie-Verlust

Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.

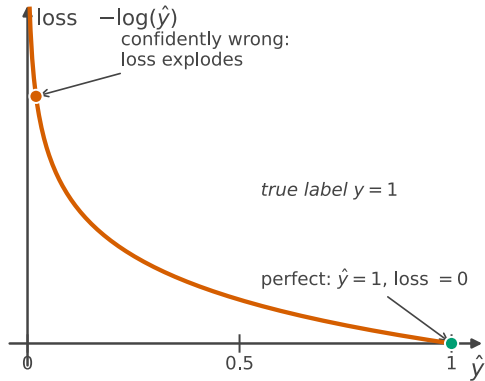
■ $\hat{y} = 1 \rightarrow \text{Verlust} = 0$.



Kreuzentropie-Verlust

Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.

- $\hat{y} = 1 \rightarrow \text{Verlust} = 0$.
- $\hat{y} \rightarrow 0 \rightarrow \text{Verlust} \rightarrow \infty$.



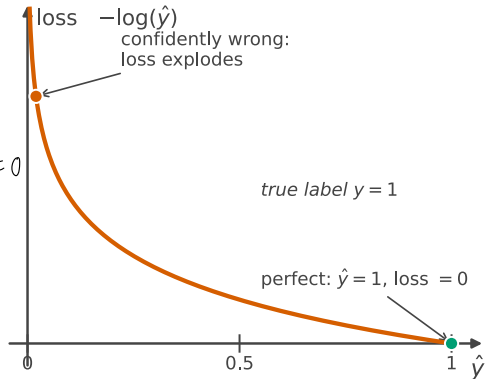
Kreuzentropie-Verlust

Ist das wahre Label $y = 1$, soll $\hat{y}$ nahe bei 1 liegen.

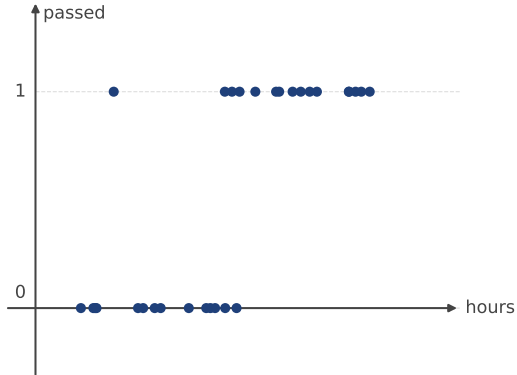
- $\hat{y} = 1 \rightarrow \text{Verlust} = 0$.
- $\hat{y} \rightarrow 0 \rightarrow \text{Verlust} \rightarrow \infty$.

Gemittelt über den Datensatz:

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

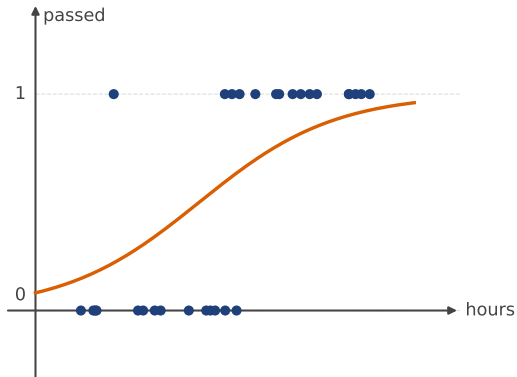


Ein Beispiel



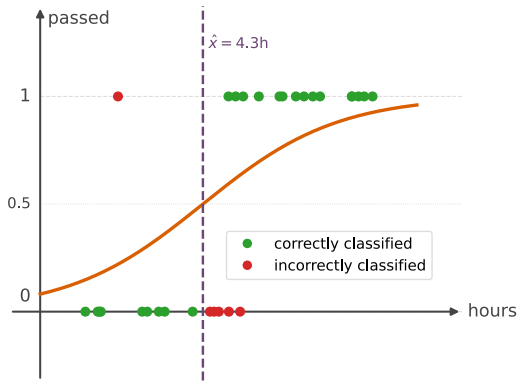
- $n = 30$ Studierende. Merkmal: gelernte Stunden. Label: bestanden ($y = 1$) oder nicht bestanden ($y = 0$).

Ein Beispiel



- $n = 30$ Studierende. Merkmal: gelernte Stunden. Label: bestanden ($y = 1$) oder nicht bestanden ($y = 0$).
- Das logistische Regressionsmodell anpassen.

Ein Beispiel



- $n = 30$ Studierende. Merkmal: gelernte Stunden. Label: bestanden ($y = 1$) oder nicht bestanden ($y = 0$).
- Das logistische Regressionsmodell anpassen.
- **Entscheidungsgrenze**: dort, wo $\hat{y} = 0.5$ gilt. Rechts davon: Vorhersage "bestanden". Links: Vorhersage "nicht bestanden".

Zusammenfassung

Logistische Regression = **lineares Modell** + **Sigmoid-Quetsche**.

Sie liefert eine **Wahrscheinlichkeit** $\hat{y} \in [0, 1]$.

Die **Entscheidungsgrenze** liegt dort, wo $\hat{y} = 0.5$ ist.

Zusammenfassung

Logistische Regression = **lineares Modell** + **Sigmoid-Quetsche**.

Sie liefert eine **Wahrscheinlichkeit** $\hat{y} \in [0, 1]$.

Die **Entscheidungsgrenze** liegt dort, wo $\hat{y} = 0.5$ ist.

- Eingesetzt für binäre Ausgänge: ja/nein, Spam/kein Spam, krank/gesund.
- Trainiert wird sie durch Minimierung des Kreuzentropie-Verlusts.

9. Theoretische Übungen

Übung 1: Linear oder logistisch?

Für jede der folgenden Vorhersageaufgaben entscheiden, ob **lineare** oder **logistische** Regression einzusetzen ist, und kurz begründen, warum.

- (a) Die morgige Temperatur in Zürich vorhersagen.
- (b) Vorhersagen, ob ein Patient Diabetes hat (ja / nein).
- (c) Den Preis eines Hauses anhand seiner Grösse und Lage vorhersagen.
- (d) Vorhersagen, ob eine Banktransaktion betrügerisch ist.
- (e) Die Abschlussnote (0–100) einer studierenden Person vorhersagen.

Übung 1: Linear oder logistisch?

Für jede der folgenden Vorhersageaufgaben entscheiden, ob **lineare** oder **logistische** Regression einzusetzen ist, und kurz begründen, warum.

- (a) Die morgige Temperatur in Zürich vorhersagen.
- (b) Vorhersagen, ob ein Patient Diabetes hat (ja / nein).
- (c) Den Preis eines Hauses anhand seiner Grösse und Lage vorhersagen.
- (d) Vorhersagen, ob eine Banktransaktion betrügerisch ist.
- (e) Die Abschlussnote (0–100) einer studierenden Person vorhersagen.

- ❗ (a) **Linear**, denn die Temperatur ist ein kontinuierlicher Wert.
- (b) **Logistisch**, denn der Ausgang ist binär (ja / nein).
- (c) **Linear**, denn der Preis ist ein kontinuierlicher Wert.
- (d) **Logistisch**, denn der Ausgang ist binär (Betrug / kein Betrug).
- (e) **Linear**, denn die Note ist ein kontinuierlicher Wert.

Übung 2: Das schlechte Modell finden

Zwei logistische Regressionsmodelle werden auf demselben Datensatz von 20 Patienten trainiert (Label: krank, ja/nein).

Modell 1 auf den *Trainingsdaten*: 0.999, 0.001, 0.998, 0.002, 0.997, 0.001, ...

Modell 2 auf den *Trainingsdaten*: 0.51, 0.49, 0.53, 0.48, 0.50, 0.52, ...

- (a) Was “sagt” jedes Modell mit seinen Ausgaben?
- (b) Welchem Modell ist auf neuen Patienten eher zu trauen, und warum?

Übung 2: Das schlechte Modell finden

Zwei logistische Regressionsmodelle werden auf demselben Datensatz von 20 Patienten trainiert (Label: krank, ja/nein).

Modell 1 auf den *Trainingsdaten*: 0.999, 0.001, 0.998, 0.002, 0.997, 0.001, ...

Modell 2 auf den *Trainingsdaten*: 0.51, 0.49, 0.53, 0.48, 0.50, 0.52, ...

- (a) Was “sagt” jedes Modell mit seinen Ausgaben?
 - (b) Welchem Modell ist auf neuen Patienten eher zu trauen, und warum?
- ❗ (a) Modell 1 ist auf jedem Trainingsbeispiel fast völlig sicher. Modell 2 ist kaum besser als ein Münzwurf.
- (b) Keines ist ideal, doch **Modell 1 ist der gefährlichere Fall**: Es hat höchstwahrscheinlich *überangepasst*, also die 20 Beispiele auswendig gelernt, statt ein allgemeines Muster zu erfassen. Auf neuen Patienten ist seine Sicherheit unzuverlässig. Modell 2 hat *unterangepasst*: Es hat kaum etwas gelernt.

Über- und Unteranpassung folgen formal in der Trainings-Lektion.

Übung 3: Die Sigmoidfunktion lesen

Ohne zu rechnen, die folgenden Fragen zur Sigmoidfunktion $\sigma(z) = \frac{1}{1+e^{-z}}$ beantworten.

- (a) Ein Modell berechnet $\mathbf{w} \cdot \mathbf{x} = 15$. Was ist $\hat{y}$ ungefähr? Welche Klasse sagt das Modell voraus?
- (b) Ein Modell berechnet $\mathbf{w} \cdot \mathbf{x} = -12$. Was ist $\hat{y}$ ungefähr?
- (c) Ein Modell berechnet $\mathbf{w} \cdot \mathbf{x} = 0$. Was ist $\hat{y}$ exakt?
- (d) Beobachtet wird $\hat{y} = 0.5$ für jeden Eingabewert im Datensatz. Was sagt das über das Modell aus?

Übung 3: Die Sigmoidfunktion lesen (Lösung)

- ❗ (a) $\hat{y} \approx 1$, das Modell sagt also Klasse 1 mit hoher Sicherheit voraus.
- (b) $\hat{y} \approx 0$, das Modell sagt also Klasse 0 mit hoher Sicherheit voraus.
- (c) $\hat{y} = 0.5$ exakt, das Modell ist also maximal unsicher.
- (d) Das Modell hat nichts gelernt: $\mathbf{w} \cdot \mathbf{x} = 0$ für jede Eingabe, d. h. alle Gewichte sind null (oder die Merkmale haben keinen Einfluss auf die Ausgabe).

Moodle quiz

mein TA kürzel: nserck